

CLAUDE.md gigante e ignorado: como curar o arquivo de contexto

Pesquisa sobre a prática "Oi! Eu já tenho um CLAUDE.md no meu projeto — até rodei o /init pra gerar —, mas sinceramente não sinto que ele ajuda muito". Como diagnosticar um CLAUDE.md que o Claude parece ignorar e reconstruí-lo enxuto para um projeto com Frontend e Backend separados.

Fernando Melo · 15 de setembro de 2026

01 · O PROBLEMA

Um CLAUDE.md gigante que o Claude parece não seguir

O ponto de partida da prática: uma dupla de devs (React no front, Node no back) já tinha um CLAUDE.md gerado pelo comando /init, mas sentia que o Claude ignorava as instruções — inclusive misturando preferências pessoais conflitantes (2 vs 4 espaços de indentação) dentro do mesmo arquivo.

Manuela: "Nossa, faz total sentido! Agora entendi melhor por que o CLAUDE.md parece ignorar as coisas — ele fica sobrecarregado e acaba priorizando o contexto da conversa."

02 · POR QUE ISSO ACONTECE

Cegueira de contexto e "Lost in the Middle"

Arquivos CLAUDE.md muito longos geram dois problemas técnicos: o imposto do token (cada linha inútil é paga em tokens de entrada em toda sessão) e a atenção diluída — o fenômeno "Lost in the Middle", em que o modelo prioriza o topo e o final do arquivo e ignora o meio quando há regras demais.

A recomendação prática que surge na pesquisa: manter o arquivo abaixo de ~200 linhas (idealmente 50-200) garante que ele caiba em um único "olhar" do modelo, com aderência praticamente total às instruções.

03 · O QUE ENTRA E O QUE NÃO ENTRA

Curadoria: o que o Claude não tem como adivinhar

ENTRA NO ARQUIVO	NÃO ENTRA NO ARQUIVO
Mecânica de comandos (npm/yarn/pnpm, comando exato de lint/test), convenções arbitrárias do time (Zod em vez de Joi, Pages Router em vez de App Router) e "zonas proibidas" — pastas ou libs que o Claude não deve tocar.	"Groselha" genérica ("escreva código limpo", "siga SOLID"), valores de segredos/chaves de API, e histórico de bugs já resolvidos na sessão — isso é papel da auto memory, não do arquivo de configuração.

Delegue a formatação, não a descreva

Como o time já usava Prettier e ESLint, a briga de indentação (2 vs 4 espaços) deixou de ser um problema de CLAUDE.md: basta apontar para a ferramenta de formatação, e o Claude lê o .prettierrc antes de escrever código — sem precisar adivinhar a preferência de cada pessoa.

- 1 Padronizem o projeto com Prettier/ESLint (ou EditorConfig).
- 2 No CLAUDE.md, apenas apontem para essa ferramenta — nunca descrevam a regra de espaçamento em texto.
- 3 Reduzam o arquivo a comandos de terminal e regras arquiteturais inegociáveis.

Modelo "Dupla Stack" (Front + Back)

Modelo de referência estudado na prática, com quatro blocos: comandos do projeto, padrões de código por camada, e regras inegociáveis de segurança/tipagem.

Estrutura efetivamente usada na prática (React + Node)

```
## Comandos do Projeto
- Instalar: npm install
- Rodar Frontend: npm run dev:front
- Rodar Backend: npm run dev:back
- Testes: npm test

## Padrões de Código e Estilo
- Formatação: seguir .eslintrc e .prettierrc
- Frontend: componentes funcionais com Hooks
- Backend: camadas (Routes -> Controllers -> Services)

## Regras Inegociáveis
- Tipagem estrita (TypeScript); proibido any
- Nunca hardcodear credenciais — usar process.env
```

Projeto (commitado) vs. preferência pessoal (local)

A solução para preferências conflitantes entre pessoas do mesmo time não é discutir dentro do CLAUDE.md do projeto, e sim usar a hierarquia de arquivos de contexto.

Camada do projeto — ./CLAUDE.md (commitado)

É lei para o time: stack, arquitetura e convenções (ex.: Tailwind, Prisma, Zod, Jest).

Camada do usuário — ~/.claude/CLAUDE.md ou CLAUDE.local.md

Preferência individual (ex.: indentação, tom de comentário), sem afetar o restante do time.

.claude/rules/ — regras aplicadas só onde importam

Para regras específicas de uma área do código (ex.: rotas do backend), a pesquisa aponta o uso de uma pasta `.claude/rules/`, com arquivos aplicados automaticamente a um caminho específico via cabeçalho "Aplica-se a".

Trecho ilustrativo simplificado

```
# Regra: rotas do backend
# Aplica-se a: src/routes/**/*

import { Router } from 'express';
import { createUserSchema } from '../schemas/userSchema';

// 1. valida payload com Zod
// 2. delega ao controller
```

Dois arquivos, um por repositório

Como frontend e backend viviam em repositórios separados, a versão final não foi um único `CLAUDE.md` genérico, e sim um arquivo por repositório: um para Next.js (Pages Router) + Tailwind, outro para Node + TypeScript + Prisma + Zod + Jest — cada um com ~20 linhas, cobrindo apenas comandos, padrões e restrições daquele lado do projeto.

Resultado: arquivo inteiro cabe em um único "olhar" do modelo, a chance de ignorar instruções cai a praticamente zero, e cada repositório só carrega o contexto que lhe pertence.

Referência externa encontrada na pesquisa

Entre os resultados relacionados sugeridos pela busca, aparece uma discussão do Reddit questionando a própria recomendação de limite: "Anthropic says keep `CLAUDE.md` under 200 lines. The most agent-heavy repos ignore this. Who's right?" — um contraponto útil para revisitar caso o projeto cresça muito.